



Complementos

Escriba sus propios complementos para extender OpenCode.

Los complementos le permiten extender OpenCode al conectarse a varios eventos y personalizar el comportamiento. Puede crear complementos para agregar nuevas funciones, integrarlos con servicios externos o modificar el comportamiento predeterminado de OpenCode.

Para ver ejemplos, consulte los [complementos](#) creados por la comunidad.

Usar un complemento

Hay dos formas de cargar complementos.

Desde archivos locales

Coloque los archivos JavaScript o TypeScript en el directorio del complemento.

``.opencode/plugins/`` - Complementos a nivel de proyecto

``${~}/.config/opencode/plugins/`` - Complementos globales

Los archivos en estos directorios se cargan automáticamente al inicio.

Desde npm

Especifique paquetes npm en su archivo de configuración.

`opencode.json`

```
{
  "$schema": "https://opencode.ai/config.json",
  "plugin": ["opencode-helicone-session", "opencode-wakatime", "@my-
org/custom-plugin"]
}
```

Se admiten paquetes npm regulares y de alcance.

Explore los complementos disponibles en el [ecosistema](#).

Cómo se instalan los complementos

Los complementos npm se instalan automáticamente usando Bun al inicio. Los paquetes y sus dependencias se almacenan en caché en `~/cache/opencode/node_modules/`.

Los complementos locales se cargan directamente desde el directorio de complementos. Para usar paquetes externos, debe crear un `package.json` dentro de su directorio de configuración (consulte [Dependencias](#)), o publicar el complemento en npm y [agregarlo a su configuración](#).

Orden de carga

Los complementos se cargan desde todas las fuentes y todos los enlaces se ejecutan en secuencia. El orden de carga es:

Configuración global (`~/config/opencode/opencode.json`)

Configuración del proyecto (`opencode.json`)

Directorio global de complementos (`~/config/opencode/plugins/`)

Directorio de complementos del proyecto (`.opencode/plugins/`)

Los paquetes npm duplicados con el mismo nombre y versión se cargan una vez. Sin embargo, un complemento local y un complemento npm con nombres similares se cargan por separado.

Crear un complemento

Un complemento es un módulo JavaScript/TypeScript que exporta uno o más complementos. funciones. Cada función recibe un objeto de contexto y devuelve un objeto de enlace.

Dependencias

Los complementos locales y las herramientas personalizadas pueden utilizar paquetes npm externos. Agregue un `package.json` a su directorio de configuración con las dependencias que necesita.

.opencode/package.json

```
{
  "dependencies": {
    "shescape": "^2.1.0"
  }
}
```

OpenCode ejecuta `bun install` al inicio para instalarlos. Luego, sus complementos y herramientas pueden importarlos.

`.opencode/plugins/my-plugin.ts`

```
import { escape } from "shescape"

export const MyPlugin = async (ctx) => {
  return {
    "tool.execute.before": async (input, output) => {
      if (input.tool === "bash") {
        output.args.command = escape(output.args.command)
      }
    },
  }
}
```

Estructura básica

`.opencode/plugins/example.js`

```
export const MyPlugin = async ({ project, client, $, directory, worktree }) => {
  console.log("Plugin initialized!")

  return {
    // Hook implementations go here
  }
}
```

La función del complemento recibe:

``project``: La información actual del proyecto.

``directory``: El directorio de trabajo actual.

``worktree``: La ruta del árbol de trabajo de git.

``client``: Un cliente SDK opencode para interactuar con la IA.

``$``: shell API de Bun para ejecutar comandos.

Soporte TypeScript

Para los complementos TypeScript, puede importar tipos desde el paquete de complementos:

my-plugin.ts

```
import type { Plugin } from "@opencode-ai/plugin"

export const MyPlugin: Plugin = async ({ project, client, $, directory,
worktree }) => {
  return {
    // Type-safe hook implementations
  }
}
```

Eventos

Los complementos pueden suscribirse a eventos como se ve a continuación en la sección Ejemplos. Aquí hay una lista de los diferentes eventos disponibles.

Eventos de comando

``command.executed``

Eventos de archivo

``file.edited``

``file.watcher.updated``

Eventos de instalación

``installation.updated``

Eventos LSP

``lsp.client.diagnostics``

``lsp.updated``

Eventos de mensajes

``message.part.removed``

``message.part.updated``

``message.removed``

``message.updated``

Eventos de permiso

``permission.asked``

``permission.replied``

Eventos del servidor

``server.connected``

Eventos de sesión

``session.created``

``session.compacted``

``session.deleted``

``session.diff``

``session.error``

```
`session.idle`
```

```
`session.status`
```

```
`session.updated`
```

Eventos de Todo

```
`todo.updated`
```

Eventos de Shell

```
`shell.env`
```

Eventos de herramientas

```
`tool.execute.after`
```

```
`tool.execute.before`
```

Eventos TUI

```
`tui.prompt.append`
```

```
`tui.command.execute`
```

```
`tui.toast.show`
```

Ejemplos

A continuación se muestran algunos ejemplos de complementos que puede utilizar para ampliar opencode.

Enviar notificaciones

Enviar notificaciones cuando ocurran ciertos eventos:

.opencode/plugins/notification.js

```
export const NotificationPlugin = async ({ project, client, $, directory,
worktree }) => {
  return {
    event: async ({ event }) => {
      // Send notification on session completion
      if (event.type === "session.idle") {
        await `$osascript -e 'display notification "Session completed!" with
title "opencode"'`
      }
    },
  }
}
```

Estamos usando ``osascript`` para ejecutar AppleScript en macOS. Aquí lo estamos usando para enviar notificaciones.

📌 NOTA

Si está utilizando la aplicación de escritorio OpenCode, puede enviar notificaciones del sistema automáticamente cuando una respuesta esté lista o cuando se produzca un error en una sesión.

Protección .env

Evite que opencode lea archivos ``.env``:

`.opencode/plugins/env-protection.js`

```
export const EnvProtection = async ({ project, client, $, directory,
worktree }) => {
  return {
    "tool.execute.before": async (input, output) => {
      if (input.tool === "read" && output.args.filePath.includes(".env")) {
        throw new Error("Do not read .env files")
      }
    },
  }
}
```

Inyectar variables de entorno

Inyecte variables de entorno en toda la ejecución del shell (herramientas de inteligencia artificial y terminales de usuario):

`.opencode/plugins/inject-env.js`

```
export const InjectEnvPlugin = async () => {
  return {
    "shell.env": async (input, output) => {
      output.env.MY_API_KEY = "secret"
      output.env.PROJECT_ROOT = input.cwd
    },
  }
}
```

Herramientas personalizadas

Los complementos también pueden agregar herramientas personalizadas a opencode:

`.opencode/plugins/custom-tools.ts`

```
import { type Plugin, tool } from "@opencode-ai/plugin"

export const CustomToolsPlugin: Plugin = async (ctx) => {
  return {
    tool: {
      mytool: tool({
        description: "This is a custom tool",
        args: {
          foo: tool.schema.string(),
        },
        async execute(args, context) {
          const { directory, worktree } = context
          return `Hello ${args.foo} from ${directory} (worktree: ${worktree})`
        },
      }),
    },
  },
}
```

El ayudante `tool` crea una herramienta personalizada a la que opencode puede llamar. Toma una función de esquema Zod y devuelve una definición de herramienta con:

`description`: Qué hace la herramienta

`args`: Esquema Zod para los argumentos de la herramienta.

`execute`: Función que se ejecuta cuando se llama a la herramienta

Sus herramientas personalizadas estarán disponibles para opencode junto con las herramientas integradas.

ⓘ NOTA

Si una herramienta de complemento utiliza el mismo nombre que una herramienta integrada, la herramienta de complemento tiene prioridad.

Registro

Utilice ``client.app.log()`` en lugar de ``console.log`` para el registro estructurado:

```
.opencode/plugins/my-plugin.ts
```

```
export const MyPlugin = async ({ client }) => {
  await client.app.log({
    body: {
      service: "my-plugin",
      level: "info",
      message: "Plugin initialized",
      extra: { foo: "bar" },
    },
  })
}
```

Niveles: ``debug``, ``info``, ``warn``, ``error``. Consulte la [documentación del SDK](#) para obtener más detalles.

Hooks de compactación

Personalice el contexto incluido cuando se compacta una sesión:

`.opencode/plugins/compaction.ts`

```
import type { Plugin } from "@opencode-ai/plugin"

export const CompactionPlugin: Plugin = async (ctx) => {
  return {
    "experimental.session.compacting": async (input, output) => {
      // Inject additional context into the compaction prompt
      output.context.push(`
## Custom Context

Include any state that should persist across compaction:
- Current task status
- Important decisions made
- Files being actively worked on
`)
    },
  }
}
```

El hook ``experimental.session.compacting`` se activa antes de que LLM genere un resumen de continuación. Úselo para inyectar contexto específico del dominio que el mensaje de compactación predeterminado omitiría.

También puede reemplazar completamente el mensaje de compactación configurando ``output.prompt``:

`.opencode/plugins/custom-compaction.ts`

```
import type { Plugin } from "@opencode-ai/plugin"

export const CustomCompactionPlugin: Plugin = async (ctx) => {
  return {
    "experimental.session.compacting": async (input, output) => {
      // Replace the entire compaction prompt
      output.prompt = `
You are generating a continuation prompt for a multi-agent swarm session.

Summarize:
1. The current task and its status
2. Which files are being modified and by whom
3. Any blockers or dependencies between agents
4. The next steps to complete the work

Format as a structured prompt that a new agent can use to resume work.
`
    },
  }
}
```

Cuando se configura `output.prompt`, reemplaza completamente el mensaje de compactación predeterminado. En este caso, se ignora la matriz `output.context`.

 [Edita esta página](#)

© Anomaly

 [Found a bug? Open an issue](#)

Última actualización: 21 may 2026

 [Join our Discord community](#)

 [Español](#)

